

INF1005 Project Report:

inSITE – Study Support for SIT Students!

Prepared By: Team P8



Chua Yu Xuan (2570298) - 2570298@sit.singaporetech.edu.sg
Ray Tan JunTao (2570291) - 2570291@sit.singaporetech.edu.sg
Lai Kaile (2570263) - 2570263@sit.singaporetech.edu.sg
Darius Koh (2570276) - 2570276@sit.singaporetech.edu.sg
Gibson Lee GuangQi (2570278) - 2570278@sit.singaporetech.edu.sg

Table of Contents (Format is not final, change if you need)

Table of Contents (Format is not final, change if you need).....	2
1. Project Overview.....	3
1.1 Project Description.....	3
1.1.1 Organisation Details.....	3
1.1.2 Project Goals.....	3
1.2 Core Functionalities.....	3
1.2.1 User Authentication and Management.....	3
1.2.2 Module Reviews.....	3
1.2.3 Community-Provided Services.....	3
1.2.4 Site Moderation.....	4
1.3 Directory Structure.....	4
2. System Design and Implementation.....	5
2.1 Architecture Overview.....	5
2.2 Frontend Design.....	5
2.2.1 Tools Used.....	5
2.2.2 Webpage Structure and Styling.....	5
2.2.3 Functionality and Integration.....	5
2.2.4 Security Considerations.....	5
2.3 Backend Design.....	5
2.3.1 Tools Used.....	5
2.3.2 Backend Architecture.....	5
2.3.3 Data Flow.....	5
2.3.4 Security Considerations.....	5
2.4 Database Design.....	5
2.4.1 Database Structure.....	5
2.4.2 Table Schema.....	5
2.4.3 Database Operations.....	5
3. Individual Contributions.....	5
4. Conclusion.....	6
Testing Credentials.....	6

1. Project Overview

1.1 Project Description

1.1.1 Organisation Details

1.1.2 Project Goals

inSite aims to provide a platform for SIT students for collaborative learning and help them prepare for the modules they will take during their time in university. By allowing students who have already taken certain modules to review said modules, they will be able to provide valuable information to newer students so that they may better prepare for any challenges faced. Furthermore, the project provides a platform where help requests may be posted, facilitating a collaborative environment where students can learn together.

1.2 Core Functionalities

1.2.1 User Authentication and Management

Users without a verified user account will still be able to view existing module reviews and help requests, but will not be able to interact (e.g. comment) with them. [requires confirmation] Account creation requires the student to provide a username, password, and their SIT student email for verification. After creating an account, the student email is used to verify the modules the student has taken to determine the account permissions. This is done through [confirm how this is done].

1.2.2 Module Reviews

A verified student account will be able to review the modules they have taken. Students can provide an overall star rating (out of 5) for the module, text content, and score ratings (out of 10) for three factors; Workload, Difficulty, and Usefulness. The date of the review is also displayed after posting to help students determine how reliable the review would be. Other students that have taken the same module will also be able to upvote, downvote or comment on a review to provide feedback on its accuracy. An inappropriate or irrelevant review can also be reported to the website admins.

1.2.3 Community-Provided Services

Students are also able to post help requests on a separate page. Students can provide a title, module, description of the request, request urgency (Low, Medium, High), a contact email, and indicate if and how much they are willing to pay for the issue to be resolved. Once a request has been fulfilled, the creator will be able to mark it as resolved.

1.2.4 Site Moderation

An account can be created as an admin account to provide moderation for the posts created on the website. An admin user is able to delete misleading or off-topic posts and reviews, [\[confirm admin perms\]](#)

1.3 Directory Structure

```
inf1005-15csc-p8/
├── docker-compose.yml
├── .env.example
├── plans/
│   └── architecture.md
├── backend/
│   ├── Dockerfile
│   ├── composer.json
│   ├── public/
│   │   └── index.php                # Slim 4 entry point
│   ├── src/
│   │   ├── App/
│   │   │   └── AppFactory.php
│   │   ├── Middleware/
│   │   │   ├── VerifiedSITizenMiddleware.php
│   │   │   ├── AdminMiddleware.php      # requires role='admin' in JWT claim
│   │   │   ├── ToxicityFilterMiddleware.php
│   │   │   ├── RateLimitMiddleware.php
│   │   │   └── RedactionMiddleware.php
│   │   ├── Controllers/
│   │   │   ├── ModuleController.php
│   │   │   ├── ReviewController.php
│   │   │   ├── ProfessorController.php
│   │   │   ├── AuthController.php
│   │   │   ├── WarRoomController.php
│   │   │   └── AdminController.php      # moderation queue, user management, audit log viewer
│   │   ├── Models/
│   │   │   ├── User.php
│   │   │   ├── Module.php
│   │   │   ├── Professor.php
│   │   │   ├── Review.php
│   │   │   ├── TrimesterOffering.php
│   │   │   └── IWSPSkill.php
│   │   ├── Services/
│   │   │   ├── AuthService.php          # register, login, JWT issue/verify, password reset
│   │   │   ├── GhostNameService.php
│   │   │   ├── SentimentService.php
│   │   │   ├── FileUploadService.php
│   │   │   └── StudyGuideExportService.php
│   │   └── Routes/
│   │       └── api.php
│   ├── config/
│   │   ├── database.php
│   │   └── jwt.php                  # JWT secret, TTL, algorithm
│   └── storage/
│       └── uploads/                # Sanitized syllabus/rubric uploads
├── frontend/
│   ├── index.html
│   ├── vite.config.js
│   └── package.json
```

```

├─ tailwind.config.js
├─ postcss.config.js
├─ manifest.webmanifest      # PWA manifest (name, icons, display, theme_color)
├─ sw.js                    # Service Worker – Workbox generated via vite-plugin-pwa
├─ src/
│   ├─ main.js
│   ├─ router.js
│   ├─ registerSW.js        # PWA service worker registration + install prompt
│   ├─ api/
│   │   ├─ modules.js
│   │   ├─ reviews.js
│   │   ├─ professors.js
│   │   └─ auth.js
│   ├─ components/
│   │   ├─ PacerGauge.js    # SVG speedometer
│   │   ├─ ProfessorSkillRadar.js # Hexagonal spider chart (responsive viewBox)
│   │   ├─ WeeklyTimeline.js # 13-week interactive timeline (horizontal scroll on mobile)
│   │   ├─ WarRoom.js       # Split-pane compare (stacked on mobile, side-by-side on md+)
│   │   ├─ DarkModeToggle.js # localStorage persistence
│   │   ├─ VibeTags.js      # SIT-specific tag taxonomy
│   │   ├─ DifficultyScatter.js # Chart.js scatter plot
│   │   ├─ BottomNavBar.js  # Mobile bottom navigation bar (visible ≤md)
│   │   └─ MobileDrawer.js  # Slide-in filter/options drawer for mobile
│   └─ pages/
│       ├─ HomePage.js
│       ├─ ModuleSearchPage.js
│       ├─ ProfessorProfilePage.js
│       ├─ ReviewFormPage.js
│       ├─ WarRoomPage.js
│       └─ PathwayPlannerPage.js
│   └─ styles/
│       ├─ main.css         # Tailwind directives + custom vars
│       └─ mobile.css       # Mobile-specific overrides (touch targets, scroll snap)
├─ nginx/
│   └─ default.conf
├─ database/
│   ├─ schema.sql
│   └─ seeds/
│       ├─ modules_seed.sql
│       └─ professors_seed.sql
└─ ...

```

2. System Design and Implementation

- The report should describe the design and implementation of the system. Include details of the tools and methodologies used by the team.

2.1 Architecture Overview

The HTML content of the website is generated by the JavaScript frontend and styles are applied using CSS. This project uses a PHP backend with a Model View Controller (MVC) based framework. API requests are routed to controllers which call functions from models to retrieve or

modify data in the database. The responses are sent back to the frontend JavaScript by the controllers for processing. Eloquent ORM through the Illuminate component is used by the backend to interact with the database, and Firebase PHP-JWT is used to implement token-based authentication.

2.2 Frontend Design

2.1.1 Tools Used

React

2.1.2 Webpage Structure and Styling

2.1.3 Functionality and Integration

2.1.4 Security Considerations

Points to yap: security should mainly be handled on backend but frontend can handle some aspects also (token storage, sanitisation, and CSP (did i use it uh))

2.3 Backend Design

2.2.1 Tools Used

To provide interaction between the backend and the MYSQL database, the Model class from Illuminate/Database/Eloquent is used to define representations of the SQL tables for the code to interact with the database. This Model class is the core of Laravel's Eloquent ORM but is used in this project without Laravel. For user and session authentication, JSON Web Token (JWT) authentication is implemented through Firebase PHP-JWT. Two types of tokens are used, Access tokens with a 15-minute lifetime for API requests and refresh tokens with a 7-day lifetime for obtaining new access tokens. **[Not Final]**

2.2.2 Backend Architecture

Key classes and functions for data handling are defined in app/Core and used throughout the backend. Database models for each table are defined in app/Models with the Eloquent ORM for interaction with the database. The SQL statements required for CRUD operations with each table are defined in app/Repositories. The classes in app/Services and app/Middleware contain functions that call the appropriate SQL statements to perform tasks. The public/index.php file is used to call the appropriate middleware or service when an API request is received from the frontend. **[Confirm final architecture]**

2.2.3 Data Flow

Incoming API requests are received by `public/index.php`, which routes the request through the required middleware (`app/Middleware`) to the corresponding controller (`app/Controllers`). The controllers call functions from the service layer (`app/Services`) that call the appropriate SQL statements from the repositories (`app/Repositories`). These statements are used to interact with the database through the Eloquent models (`app/Models`). **This might be too repetitive**

2.2.4 Security Considerations

Server-side sanitization + validation (`app/Services/VerificationService.php`)

2.4 Database Design

2.3.1 Database Structure

2.3.2 Table Schema

2.3.3 Database Operations

3. Individual Contributions

- Each team member should list in point form the parts of the website that they worked on.

IDK how y'all want to do this one

4. Conclusion

Testing Credentials

- All credentials needed for logging into the website (as customer, admin, employee, etc.) for testing & assessment purposes should be listed in the report. Be sure that these credentials will work with *just username + password*, without need for 2FA or any external authentication (e.g. email or SMS).

user credentials:

Username:

Password:

admin credentials:

Username:

Password: